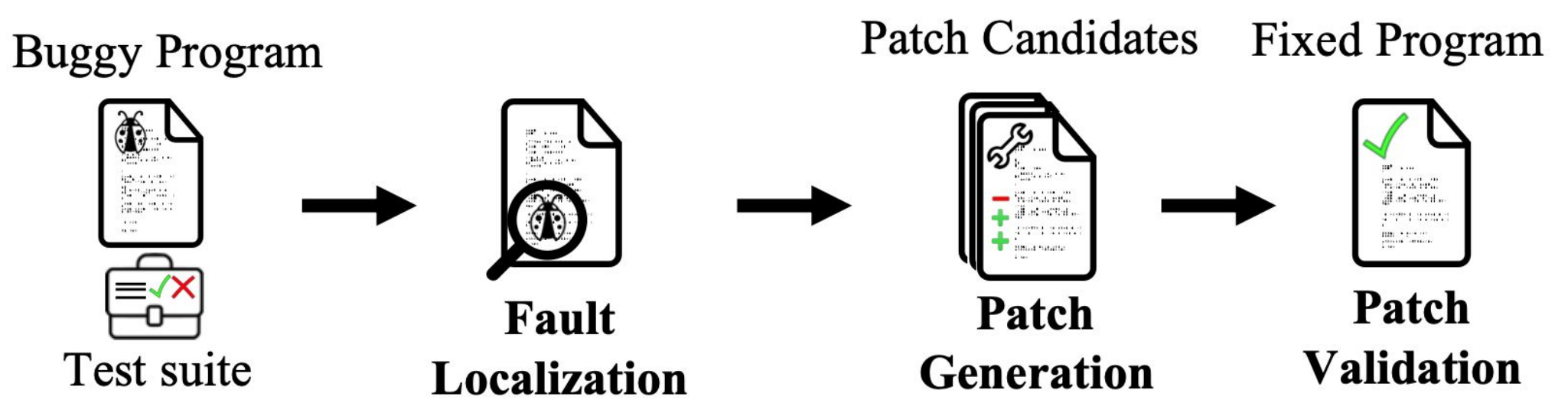


Abstract

In an increasingly digital world, code has become an integral part of everyday life for hundreds of millions of people. Repairing bugs is a crucial aspect of code development because unfixed bugs can cause a wide range of consequences, from unexpected system crashes to cyberattacks. As a result, billions of dollars and vast amounts of time are spent on code debugging every year. To reduce the burden of code repair on developers, this research introduces a new multi-agent large language model system to automatically repair bugs. Through specialized prompts, knowledge extraction from databases, and a collaborative framework, the large language model agents are able to repair multi-location bugs: a complex but prevalent bug pattern in real-world code. An evaluation on the Defects4J v1.2 dataset reveals that the system significantly outperforms state-of-the-art, traditional, and deep learning baselines with reasonable time and cost. This repair system lays the foundation for a promising new approach to automated program repair which can be applied to both academic settings and large-scale production environments in the future.

Background

Automated program repair is the process of automatically fixing software bugs with little to no human intervention. A **patch** is a piece of code that modifies a program to fix a bug.



Liu et al. 2020 summarized this workflow of traditional automated program repair.² In recent years, researchers have become less reliant on test suites as they are time-consuming to run and may not be comprehensive. This research does not utilize test suites.

Multi-location bugs are bugs that span multiple non-contiguous locations in source code.

Defects4J Dataset

Defects4J is the most popular bug repair dataset. The most used version, Defects4J v1.2, spans 395 bugs across six real-world open-source projects. Of the 395 bugs, 190 are multi-location. In this research, all analysis and testing is performed on these 190 bugs. As defined by previous works, a patch is correct if it is syntactically or semantically equivalent to the human patch.

```
1 commit dd9f3710757cc652f2681fba7ac28e1845e0a31a
2 diff --git a/.../Mockito.java b/.../Mockito.java
3 @@ -824,7 +824,7 @@ public static <T> T mock(Class<T> classToMock, Answer defaultAnswer
4     ) {
5         public static <T> T mock(Class<T> classToMock, MockSettings mockSettings) {
6             ~ return Mockito.CORE.mock(classToMock, mockSettings);
7         + return Mockito.CORE.mock(classToMock, mockSettings, true);
8     }
9 diff --git a/.../MockitoCore.java b/.../MockitoCore.java
10 @@ -20,10 +29,11 @@ public class MockitoCore {
11     ~ public <T> T mock(Class<T> classToMock, MockSettings mockSettings, boolean
12         shouldResetOngoingStubbing) { return mock(classToMock, mockSettings); }
13 + public <T> T mock(Class<T> classToMock, MockSettings mockSettings) {
14     + public <T> T mock(Class<T> classToMock, MockSettings mockSettings, boolean
15         shouldResetOngoingStubbing) {
16         ~ mockingProgress.validateState();
17         + if (shouldResetOngoingStubbing) {
18             ~ mockingProgress.resetOngoingStubbing();
19         }
```

Chart 5
◀ Previous Bug | Next Bug ▶

Patterns
Conditional block addition with return statement Logic expression reduction

Actions
Conditional (if) branch addition Conditional expression reduction Method call addition Return statement addition

Patched by
Cardumen DeepRepair DynaMoth GPLL GenProg JKaI Nopol

Human Patch
source/org/jfree/data/xy/XYSeries.java CHANGED

```
@@ -541,11 +541,15 @@ public XYDataItem addOrUpdate(Number x, Number y) {
541 541     if (x == null) {
542 542         throw new IllegalArgumentException("Null 'x' argument.");
543 543     }
544 +     if (this.allowDuplicateValues) {
545 +         add(x, y);
546 +         return null;
547 +     }
548 ~ // If we get to here, we know that duplicate X values are not permitted
549 ~ XYDataItem overwritten = null;
550 ~ int index = indexOf(x);
551 ~ if (index >= 0 && this.allowDuplicateValues) {
552 +     if (index >= 0) {
549 553         XYDataItem existing = (XYDataItem) this.data.get(index);
550 554         try {
551 555             overwritten = (XYDataItem) existing.clone();
```

Examples of multi-location bugs (top: Mockito 16, bottom: Chart 5). Code acquired from Defects4J.¹ The bottom image depicts the web user interface which contains additional information such as patch patterns, actions, and repair tools that have already patched the code.

Patching Multi-Location Software Bugs: A Multi-Agent Large Language Model Framework for Automated Program Repair

System Design and Methodology

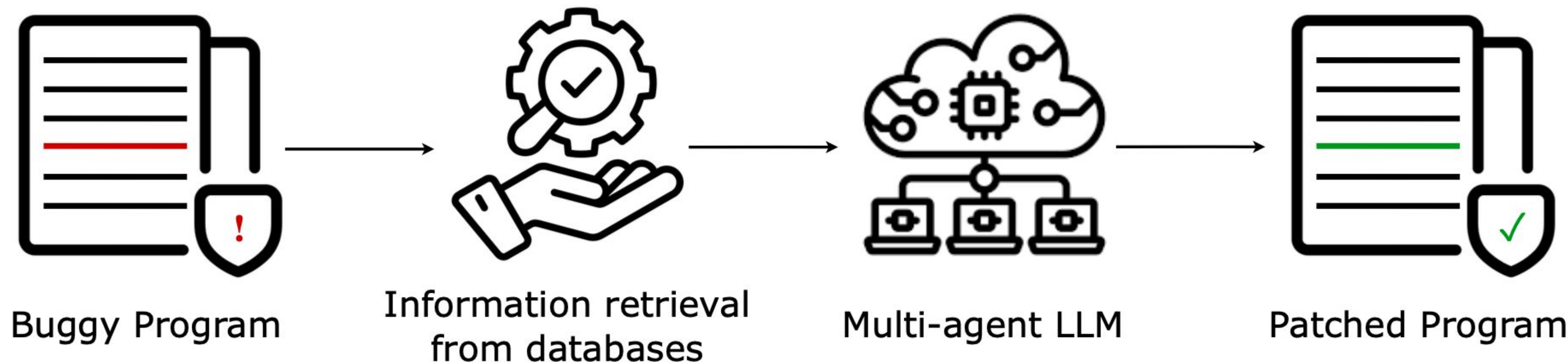


Figure 1: Overall system architecture. Figure created by researcher using draw.io (2024).

This repair system first takes the source code and bug locations as input. Then, it retrieves candidate APIs and multi-location repair patterns from the databases. Finally, the task is distributed to six large language model (LLM) agents which are instructed to collaboratively generate, verify, and return the best patch.

Challenge #1: Identifying and applying prior knowledge. While LLMs have been trained on extensive open-source code bases, they struggle to apply their general programming knowledge to specific bug repair scenarios.

API database: An API database is constructed to help the LLM agents rapidly pinpoint candidate APIs to call. This database contains the most commonly used APIs in Defects4J programs. It includes general APIs (i.e., in at least 10% of all programs) and project-specific APIs (i.e., in at least 50% of all programs under one project).

API Purpose	Count	Examples
Collection	11	java.util.List, com.google.common.collect.Sets
Javascript handling	5	com.google.common.base.Preconditions
Input/output (I/O) handling	3	java.io.Serializable, java.io.IOException
Built-in classes	6	java.lang.Character, java.lang.Integer
Miscellaneous	6	java.util.Locale, org.joda.time.field.FieldUtils

Table 1: 31 total APIs classified by purpose

Repair pattern database: A database summarizing the most common repair patterns in Defects4J is constructed. The pattern descriptions and corresponding step-by-step repair instructions serve as a guide for the LLM agents as they fill in the framework with program-specific details.

	Variable Addition	Similar If-Check	Method Implementation	New Method	API Importation	Independent Fixes	Other
Chart	2	6	0	0	0	2	1
Closure	13	4	12	5	0	16	5
Lang	5	8	9	1	0	4	6
Math	16	10	27	1	2	6	2
Mockito	2	0	7	8	10	0	0
Time	6	5	3	0	0	0	1
Total	44 (23%)	33 (17%)	58 (31%)	15 (8%)	12 (6%)	28 (15%)	15 (8%)

Table 2: Multi-location repair pattern distribution by project

Challenge #2: Length of LLM prompt. When LLMs are given excessively long prompts, they often lose context and fail to follow all instructions due to their limited long-term memory.

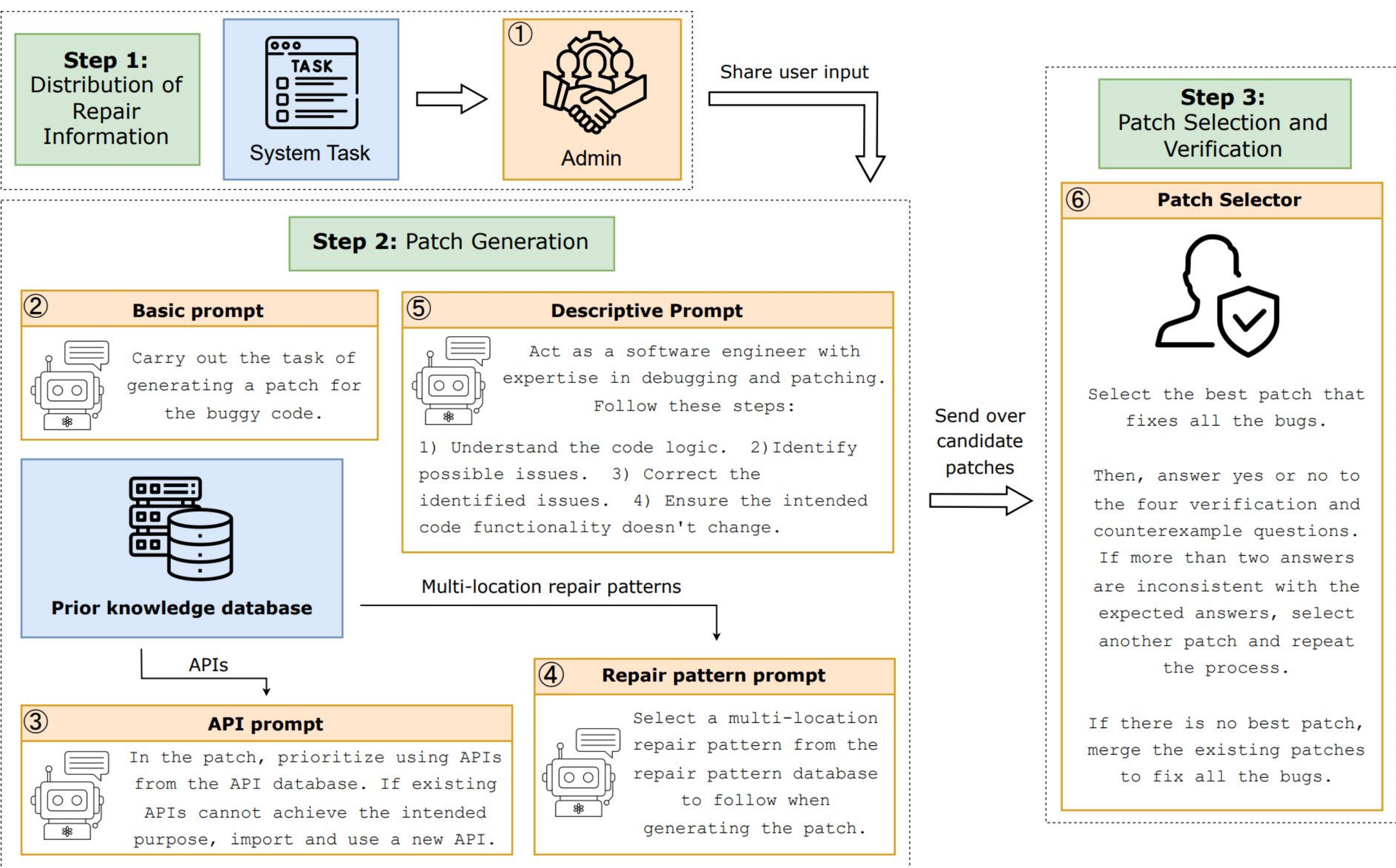
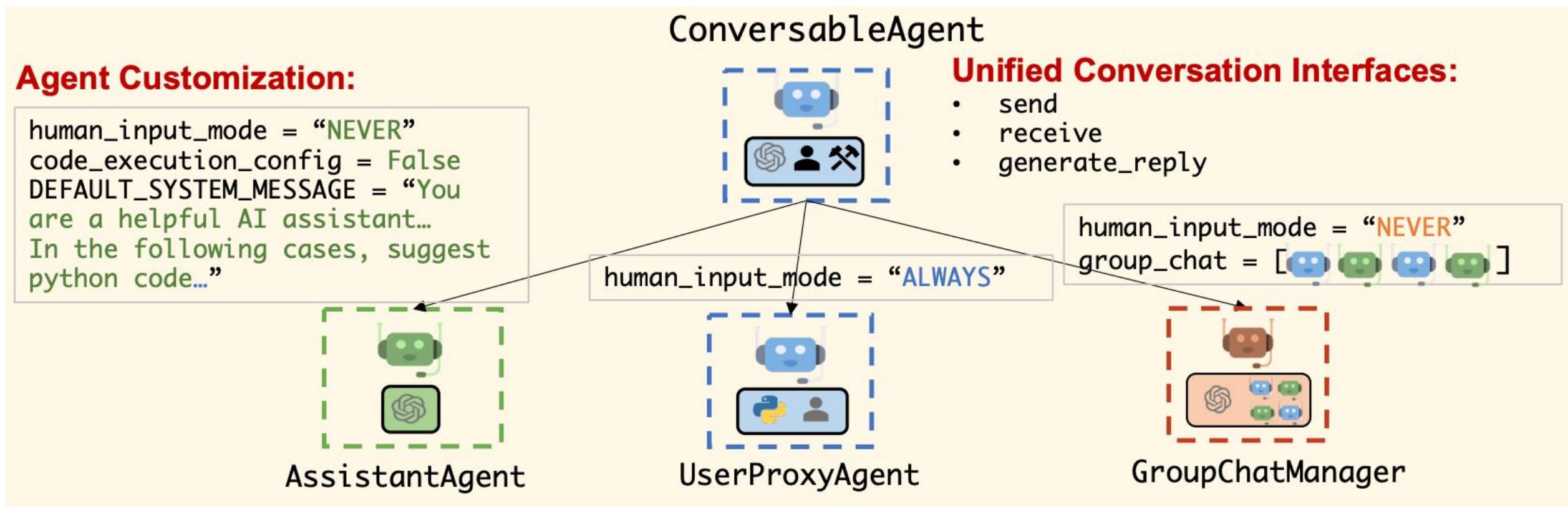


Figure 2: Overview of multi-agent interaction. The natural language portion of the prompts have been simplified in this figure. Figure created by researcher using draw.io (2024).

Implementation

The system is implemented with AutoGen, an open-source multi-agent LLM framework. I modified the configurations of the built-in agents and implemented additional functions as needed. The prior knowledge databases were imported as separate files, and prompts were provided via natural language. GPT-4 was set as the model for each agent. This system operates under the assumption of perfect fault localization (i.e., the bug location has already been identified). This is the preferred setting for recent works as it allows for pure analysis of the patch generation technique.



Wu et al. 2023 proposed the following built-in AutoGen agents, which can be modified and have unified conversation interfaces.⁴

Results

Project	Count	This research	GPT-4	Nopol	DynaMoth	DeepRepair
Chart	10	7	3	3	3	5
Closure	54	20	5	20	19	0
Lang	31	13	4	0	1	4
Math	59	23	12	10	9	18
Mockito	22	5	1	0	0	0
Time	14	2	0	1	0	0
Total	190	70 (37%)	25 (13%)	34 (18%)	32 (17%)	27 (14%)

Table 3: Repair results on all multi-location bugs compared to base GPT-4 and traditional repair tools.

This research yields a significant increase in accuracy compared to base GPT-4 and traditional repair tools. Moreover, when compared to the 30 publicly available results of existing multi-location repair tools (i.e., ITER, Hercules, VarFix, DEAR), this research yields a 23%, 78%, 220%, and 700% increase in repair accuracy, respectively. Note that DEAR does not assume perfect fault localization, hence the low repair accuracy.

	Total	Average
API Requests	2,054	10.81
Token consumption	5,842,074	30,747.76
Time (seconds)	18,663	98.23
Cost (dollars)	80.25	0.42

Table 4: Cost analysis.

Compared to current LLM-based repair tools (i.e., RepairAgent, ITER, ChatRepair), the system proposed in this research incurs an approximately equivalent monetary cost per generated patch and is especially efficient in terms of time consumption.

Impact

- This research is one of the first to utilize a collaborative multi-agent framework to specifically target multi-location bug repair.** It outperforms state-of-the-art traditional and LLM-based repair tools with reasonable time and cost. This paves the way for a promising new approach to LLM-based repair, which the automated program repair community can extend and further refine going forward.
- This system can be incorporated into continuous integration and continuous deployment (CI/CD) pipelines to simplify software development.** Currently, most bugs are manually repaired by developers when testing failures occur. When used with a fault localization tool, this system can streamline the CI process by fully automating code repair, making it quicker and easier to merge bug-free code into a centralized code base for deployment.
- The multi-agent framework can be easily adopted and extended.** The current system can be used to directly repair Java code. It can also be treated as a general framework that can be flexibly modified and extended to other programming languages and bug types.
- The generated patches can be used as training data.** The buggy source code files, combined with the labeled candidate patches, can be used to train and fine-tune code generation models and large language models in the future.

Selected References

[1] Defects4j. 2024. A Database of Real Faults. <https://program-repair.org/defects4j-dissection/#/>.
[2] Liu, Kui, et al. "On the efficiency of test suite based program repair: A systematic assessment of 16 automated repair systems for java programs." *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*. 2020.
[3] "What Is CI/CD and CI/CD Pipeline? - Processes, Stages, Benefits." *OpsMx Blog* |, 20 Dec. 2024, www.opsmx.com/blog/what-is-a-ci-cd-pipeline/.
[4] Wu, Qingyun, et al. "Autogen: Enabling next-gen llm applications via multi-agent conversation framework." *arXiv preprint arXiv:2308.08155* (2023).